Encoders

Tópicos em Fundamentos e Métodos da Computação

Prof. Dr. Luís Paulo Faina Garcia

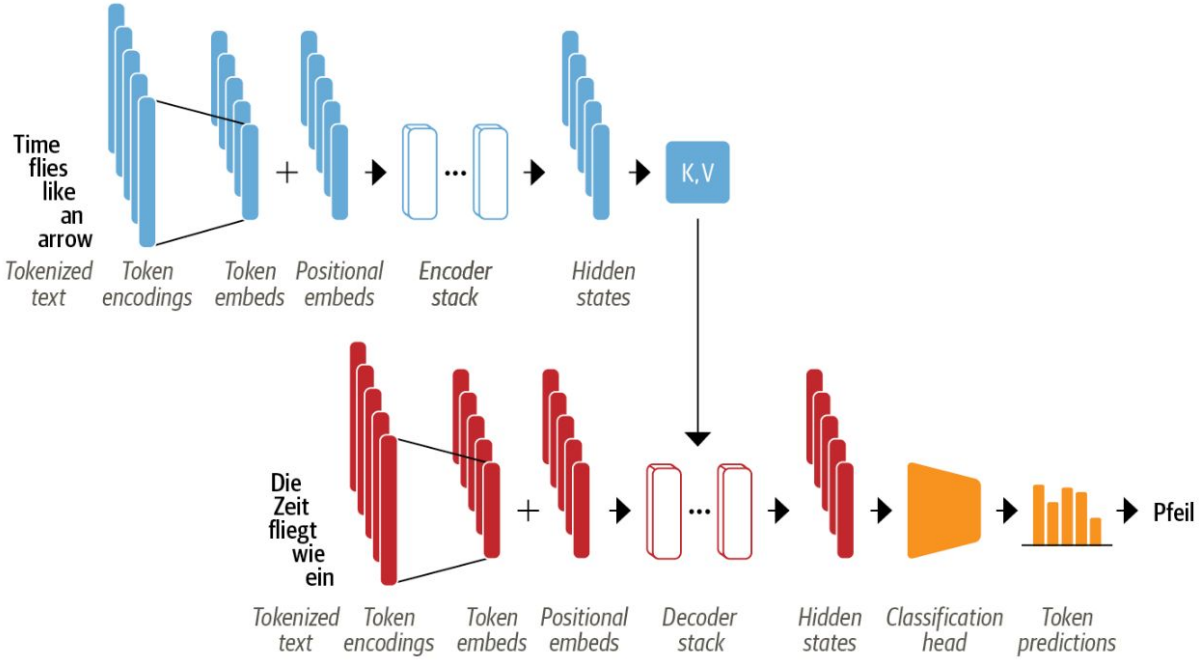
Universidade de Brasília
Departamento de Ciência da Computação

Transformer Architecture

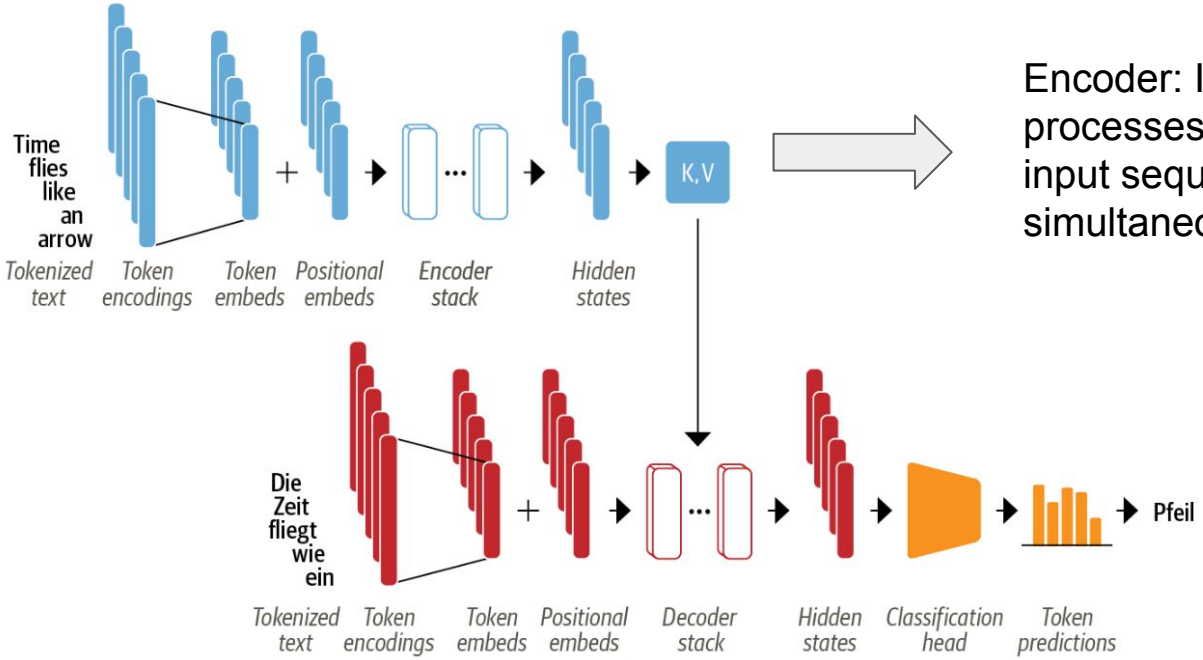
Originally designed for sequence-to-sequence tasks (like machine translation). This architecture consists of two components:

- **Encoder:** Converts input tokens into a sequence of embedding vectors (hidden state/context);
- **Decoder:** Uses encoder state to generate output tokens.

Transformer Architecture

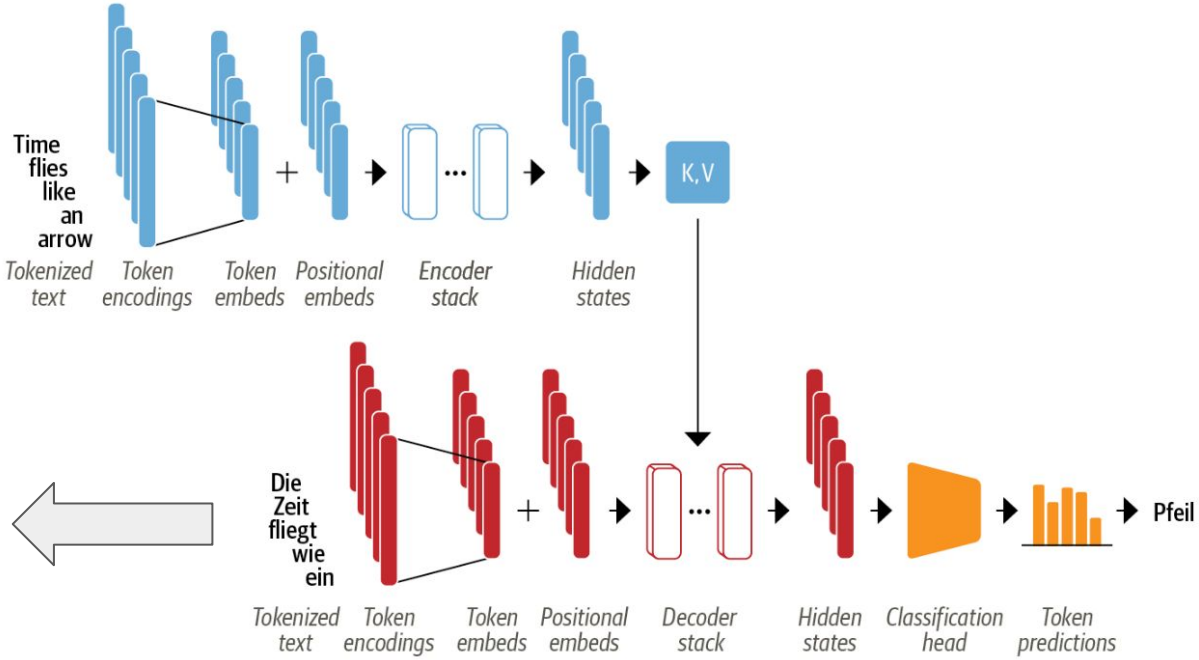


Transformer Architecture



Encoder: It processes the entire input sequence simultaneously

Transformer Architecture



Decoder: Uses the encoder's hidden state to iteratively generate an output sequence

Transformer Architecture

1. The input text is tokenized and converted to token embeddings;
2. The token embeddings are thus combined with positional embeddings that contain positional information for each token;
3. The encoder is composed of a stack of encoder layers or blocks. The same is true of the decoder, which has its own stack of decoder layers;
4. The encoder output is fed to each decoder layer, and the decoder then generates a prediction for the most probable next token in the sequence;
5. The output of this step is then fed back into the decoder to generate the next token, and so on until the decoder predicts the EOS token or we reached a maximum length.

Transformer Architecture

The Transformer architecture was originally designed for sequence-to-sequence tasks like machine translation, but both the encoder and decoder blocks were soon adapted as standalone models.

Encoder-only

- The encoder-only models convert an input sequence of text into a rich numerical representation;
- The representation computed for a given token in this architecture depends both on the left (before the token) and the right (after the token) contexts.
- This is often called bidirectional attention.

Encoder-only

- That is well suited for tasks like:
  - Text Classification
  - Named Entity Recognition
  - and others …
- Examples:
  - BERT (Bidirectional Encoder Representations From Transformers)
  - RoBERTa e DistilBERT, ModernBERT
  - BERTimbau, ModBERTBr e BERTomelo
  - …

Bertram "Bert" (born July 26, 1969)
from Sesame Street

Decoder-only

- The decoder-only models will auto-complete the sequence by iteratively predicting the most probable next word;
- The representation computed for a given token in this architecture depends only on the left context.
- This is often called causal or autoregressive attention.

Decoder-only

- Examples:
  - GPT (Generative Pretrained Transformers)
  - Gemini
  - Claude Sonnet
  - LLama
  - Qwen
  - …

Encoder-decoder

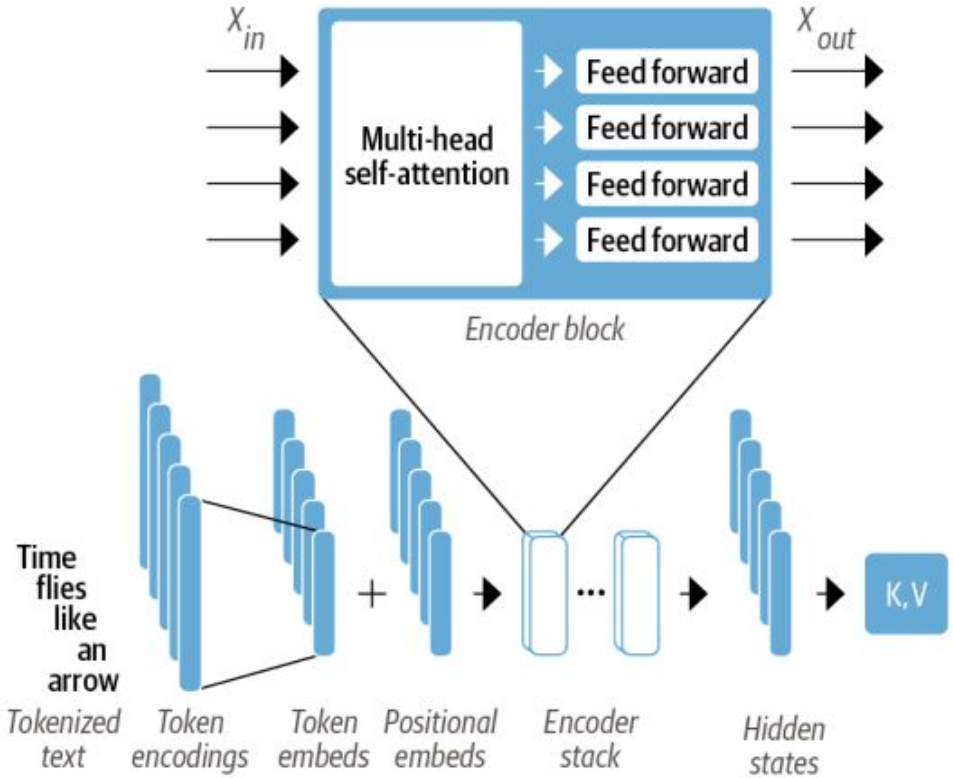
- The encoder-decoder are used for modeling complex mappings from one sequence of text to another;
- Suitable for machine translation and summarization tasks;
- Examples:
  - BART
  - T5
  - …

Zooming the Encoder

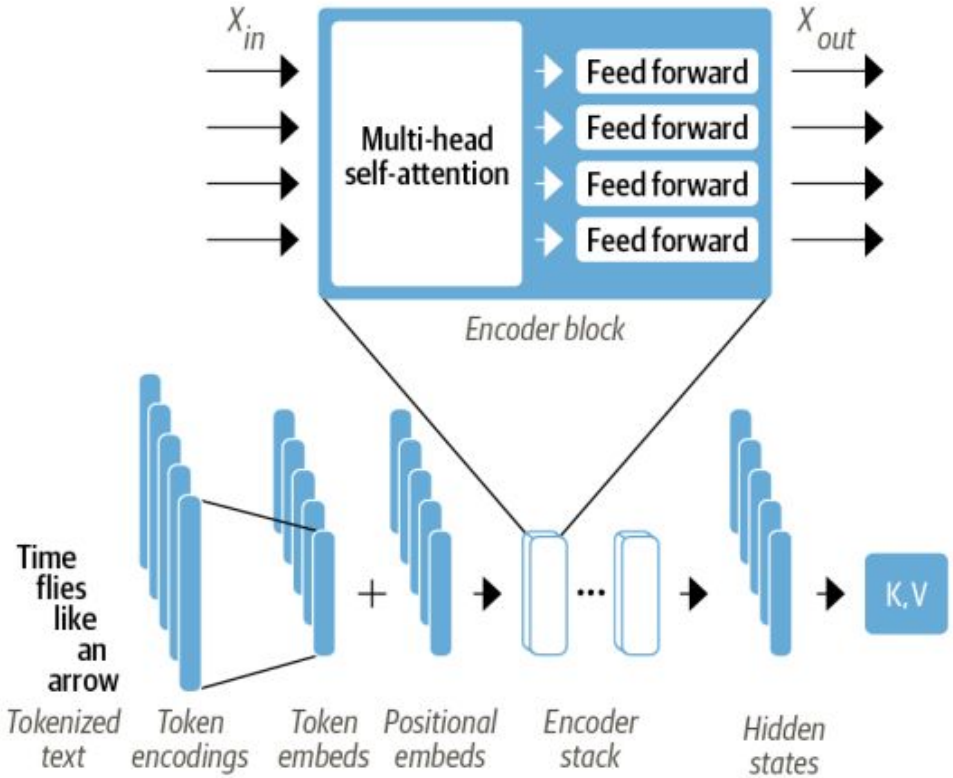
The transformer encoder consists of many encoder layers stacked next to each other. Each encoder layer receives a sequence of embeddings and feeds them through the following sublayers:

- A multi-head self-attention layer
- A fully connected feedforward layer that is applied to each input embedding

Zooming the Encoder



Zooming the Encoder



+ skip connections and layer normalization

From Text to Tokens

- Transformer models cannot process raw text directly;
- They require numerical input (vectors);
- Pipeline:
  - Raw text → Tokenization
  - Tokens → Embeddings
  - Embeddings → Transformer layers

From Text to Tokens

- Tokenization is the process of splitting text into smaller units called tokens. The quality of tokenization directly impacts model performance.
- Examples:
  - Sentence: "Transformers are powerful models"
  - Word-level → "Transformers", "are", "powerful", "models"
  - Subword-level → "Transform", "ers", "are", "power", "ful", "models"

From Text to Tokens

- Why not just words?
  - Large vocabularies are inefficient
  - Unknown words (out-of-vocabulary problem)

Types of Tokenizers

- Word-based Tokenization
  - Splits text by spaces
  - Cannot handle unknown words well
  - Large vocabulary required
- Character-based Tokenization
  - Splits text into individual characters
  - Generates very long sequences
  - Harder to learn meaning

Types of Tokenizers

- Subword Tokenization
  - Byte Pair Encoding (BPE): Iteratively merges frequent character pairs
  - WordPiece: Builds tokens based on likelihood
  - SentencePiece: Works directly on raw text (no pre-tokenization)

Special tokens

- [CLS] → Sequence representation (used for classification)
- [SEP] → Separates sentences / marks boundaries
- [UNK] → Unknown / out-of-vocabulary token
- [PAD] → Padding for equal-length batches
- [MASK] → Used in masked language modeling(encoders)
- [BOS] → Beginning of sequence (decoders)
- [EOS] → End of sequence (decoders)

From Tokens to Embeddings

- Each token is mapped to a vector (embedding);
- Embeddings are learned during training;
- What is captured?
  - Semantic meaning
  - Syntactic behavior
  - Contextual usage (in deeper layers)
- Types of Embeddings in Transformers:
  - Token embeddings
  - Positional embeddings
  - Segment embeddings

Self-Attention

- Attention is a mechanism that allows neural networks to assign a different amount of weight or “attention” to each element in a sequence;
- For text sequences, the elements are token embeddings where each token is mapped to a vector of some fixed dimension;
- In self-attention, weights are computed among elements within the same sequence.

Self-Attention

The main idea behind self-attention is that instead of using a fixed embedding for each token, we can use the whole sequence to compute a weighted average of each embedding.

Self-Attention

Another way to formulate this is to say that given a sequence of token embeddings $x_1, ..., x_n$, self-attention produces a sequence of new embeddings $x_1', ..., x_n'$ where each x'_i is a linear combination of all the x_j:

$$x'_i = \sum_{j=1}^{n} w_{ji} x_j$$

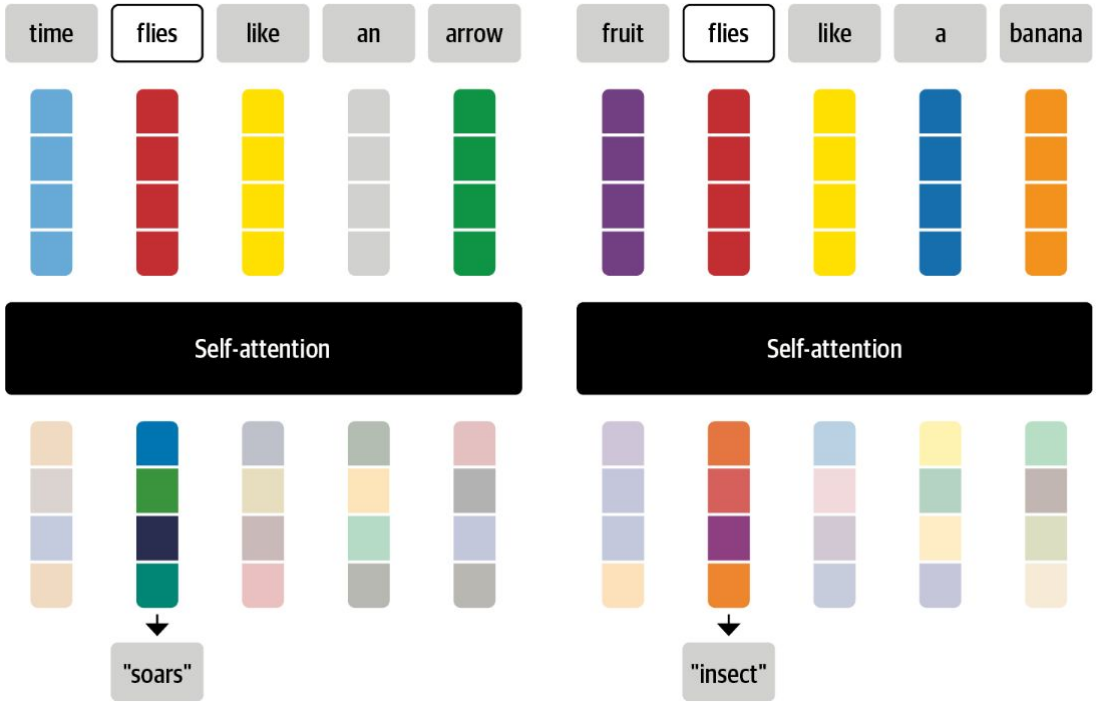
Self-Attention

That given a sequence of token embeddings $x_1, ..., x_n$, self-attention produces a sequence of new embeddings $x_1', ..., x_n'$ where each x'_i is a linear combination of all the x_j:

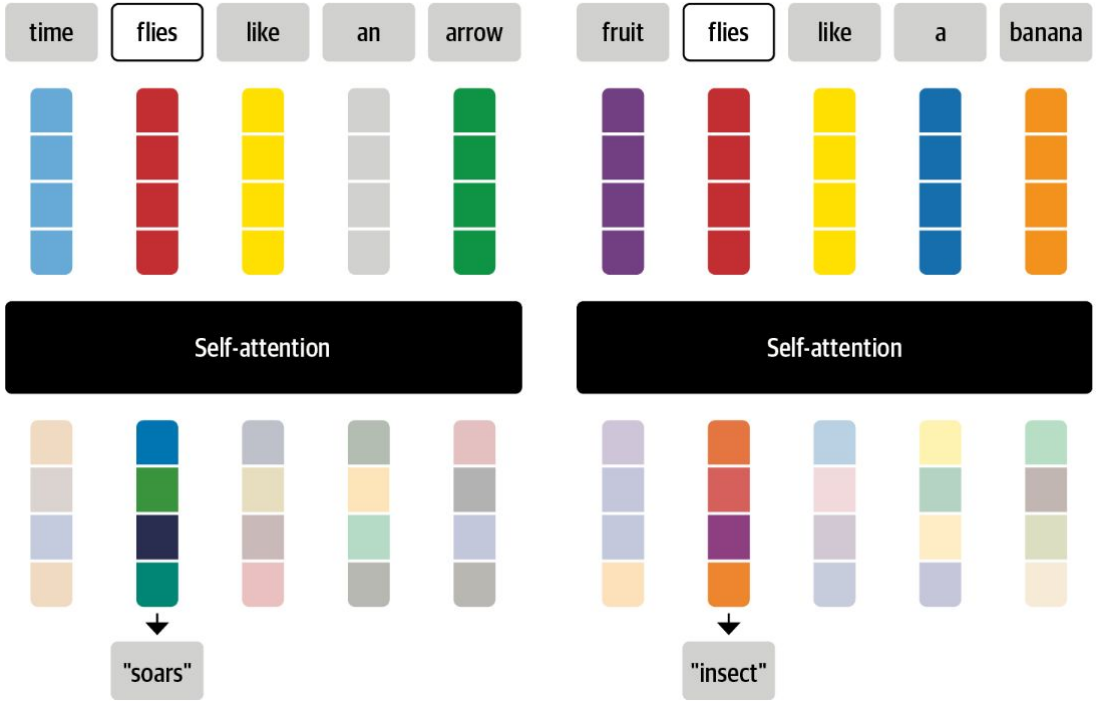
$$x'_i = \sum_{j=1}^{n} w_{ji} x_j$$

The coefficients w_{ji} are called attention weights

Self-Attention



Self-Attention



Embeddings that are generated in this way are called contextualized embeddings.

Scaled dot-product attention

There are several ways to implement a self-attention layer, but the most common one is scaled dot-product attention. There are four main steps required to implement this mechanism:

1. Project each token embedding into three vectors called query, key, and value;

2. Compute attention scores. We determine how much the query and key vectors relate to each other using the scaled dot-product attention;

Scaled dot-product attention

3. Compute attention weights and then normalize with a softmax to ensure all the column values sum to 1. The result is a $n \times n$ matrix now contains all the attention weights w_{ji};

4. Update the token embeddings. Once the attention weights are computed, we multiply them by the value vector $v_1, \ldots, v_n$ to obtain an updated representation for embedding $x'_i = \sum_j w_{ji} v_j$.

Demystifying Queries, Keys, and Values

Dishes recepe

Query

Scan the shelves

Keys

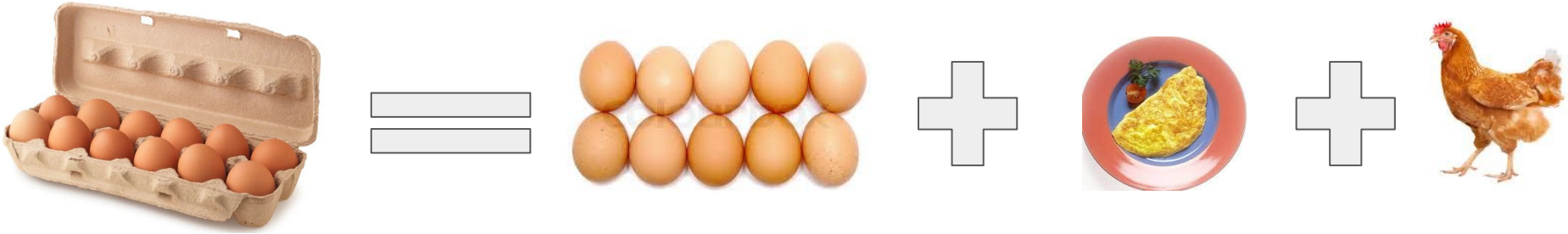
Values

Demystifying Queries, Keys, and Values

Self-attention is a more abstract and “smooth” version of this: every label in the supermarket matches the ingredient to the extent to which each **key** matches the **query**. So if your list includes a dozen eggs, then you might end up grabbing 10 eggs, an omelette, and a chicken wing.

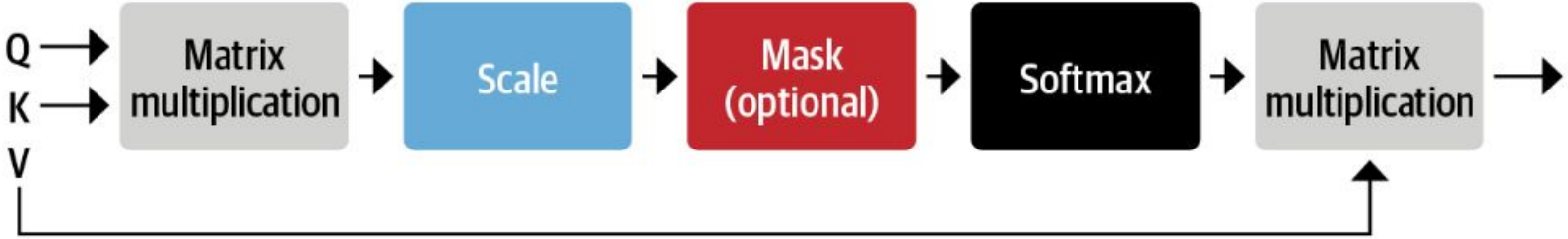
Demystifying Queries, Keys, and Values

Self-attention is a more abstract and “smooth” version of this: every label in the supermarket matches the ingredient to the extent to which each **key** matches the **query**. So if your list includes a dozen eggs, then you might end up grabbing 10 eggs, an omelette, and a chicken.



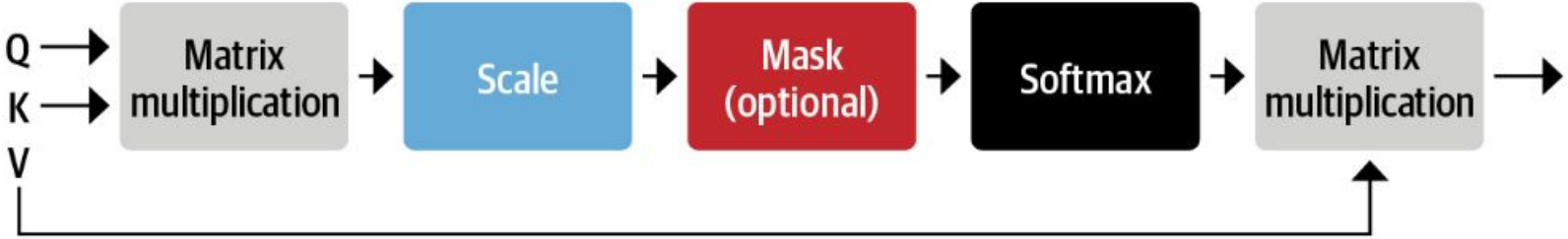
Scaled dot-product attention

The diagram of operations to compute scaled dot-product attention:



Scaled dot-product attention

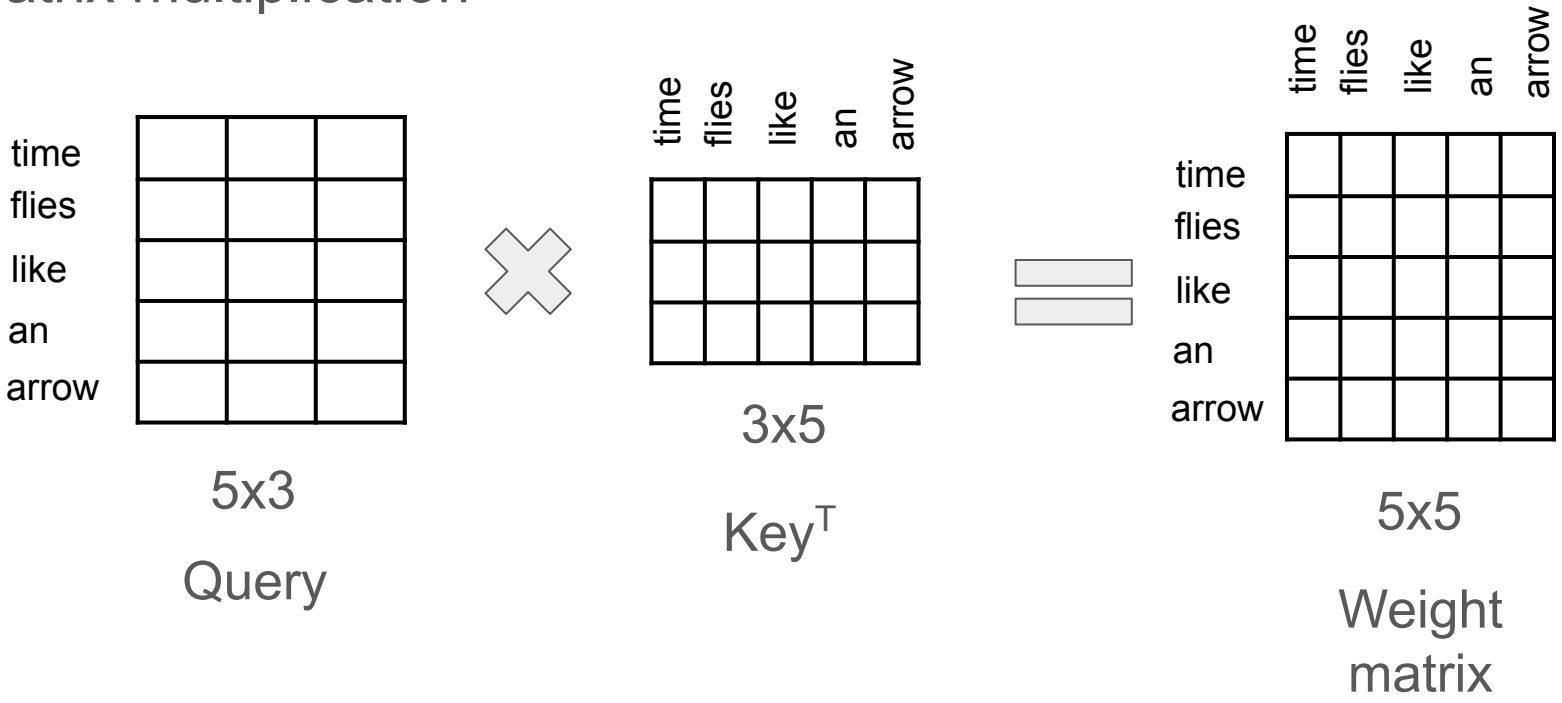
The diagram of operations to compute scaled dot-product attention:



$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

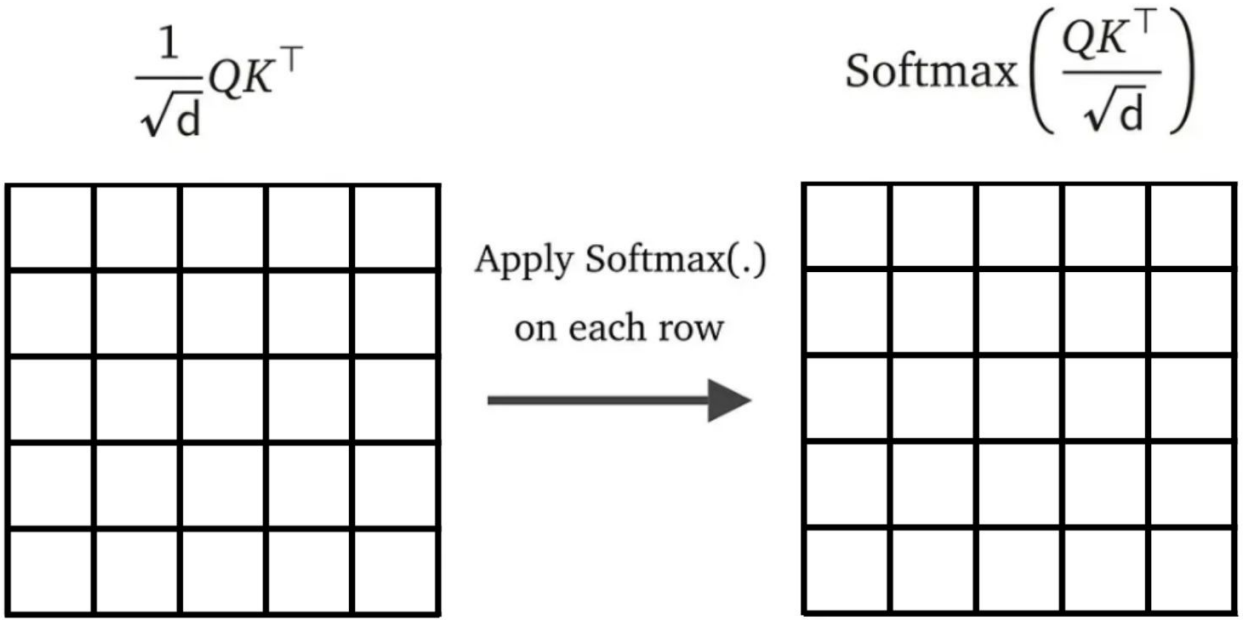
Attention Mechanism without Trainable Parameters

Matrix multiplication



Attention Mechanism without Trainable Parameters

Scale + softmax



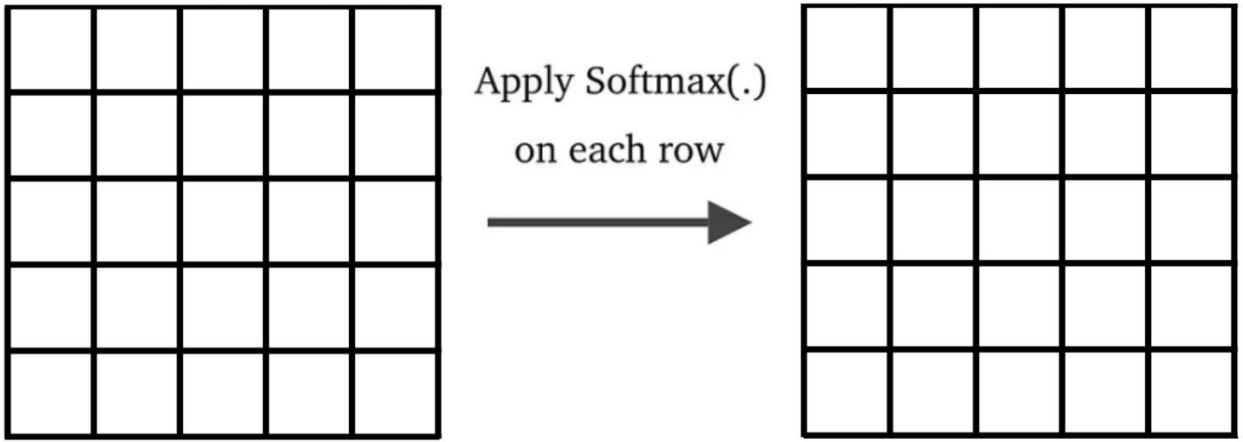
Attention Mechanism without Trainable Parameters

Scale + softmax

Without this adjustment, the dot product results in very large values.

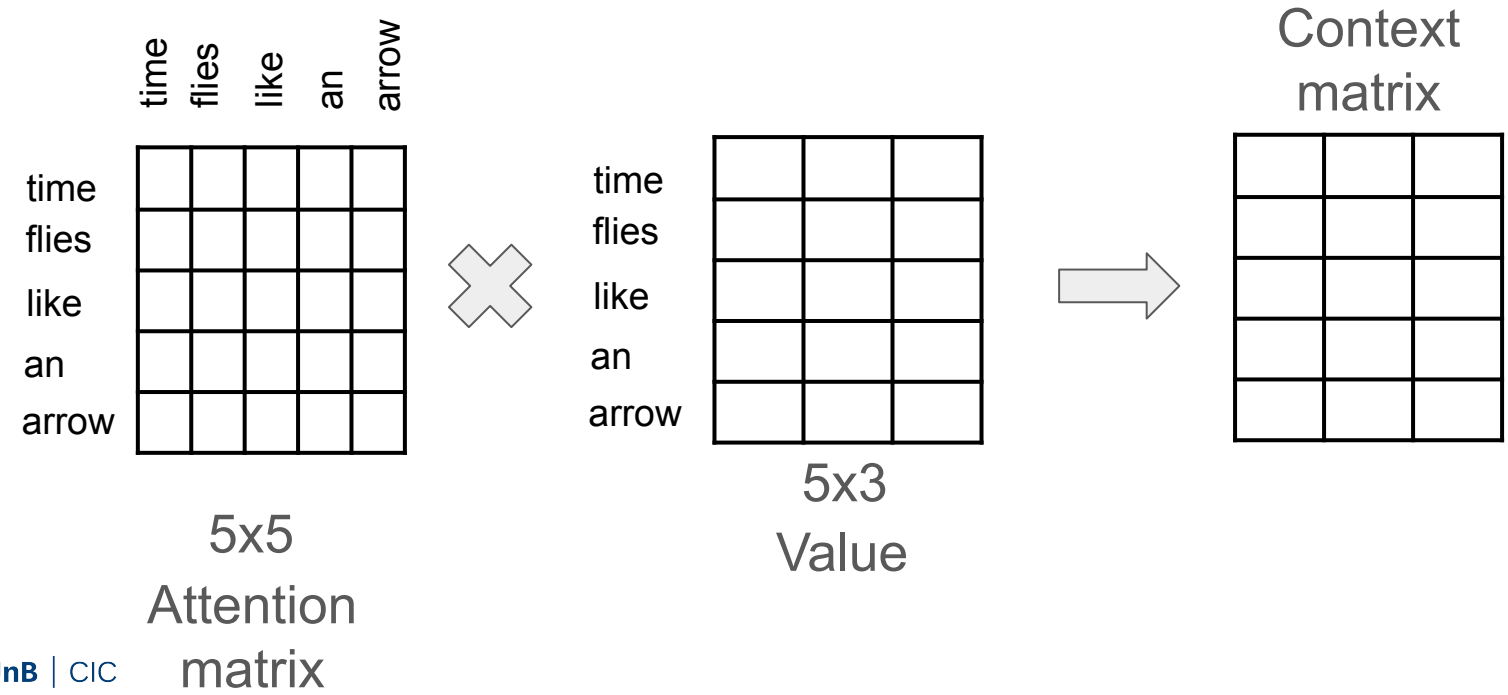
$$\frac{1}{\sqrt{d}} QK^\top$$

$$\text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)$$



Attention Mechanism without Trainable Parameters

Matrix multiplication

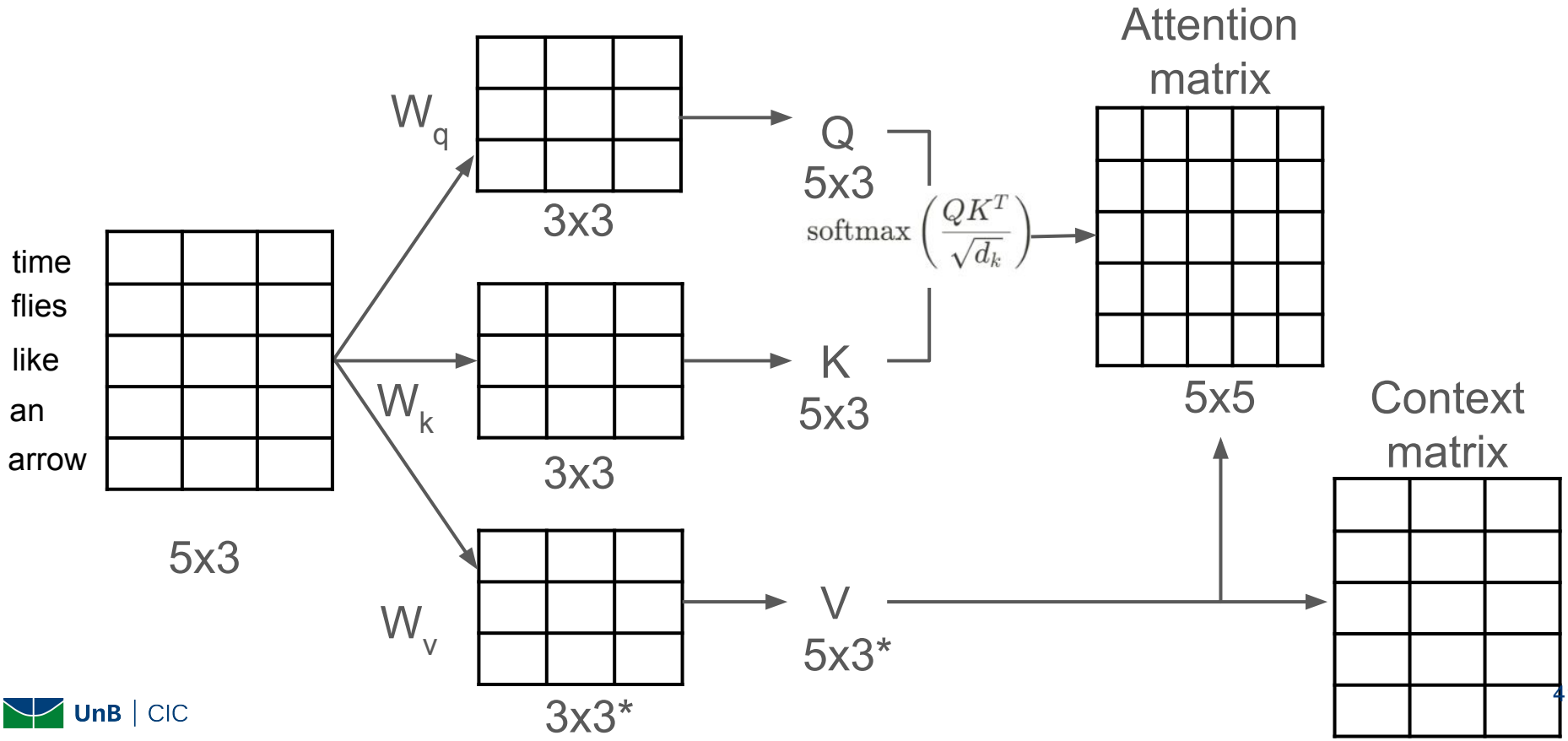


Attention Mechanism with Trainable Parameters

Query, Key, Value matrices are not just direct copies of word embeddings. Instead, they are learned transformations of these embeddings. To achieve this, we introduce three trainable weight matrices:

- W_q: Learns how to project word embeddings into the query space.
- W_k: Learns how to project word embeddings into the key space.
- W_v: Learns how to project word embeddings into the value space.

Attention Mechanism with Trainable Parameters

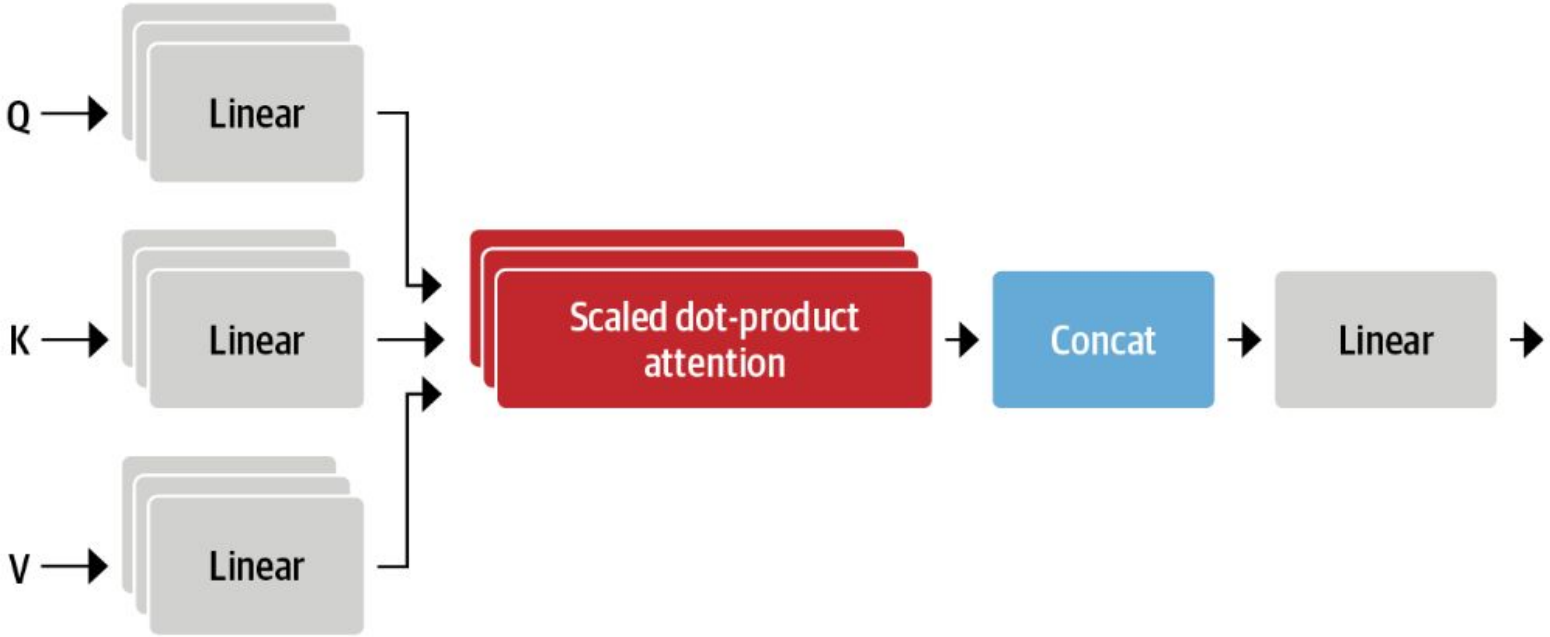


Multi-headed attention

Do we need more than one attention head?

- One head tends to focus on mostly one aspect of similarity. Having several heads allows the model to focus on several aspects at once;
- For instance, one head can focus on subject-verb interaction, whereas another finds nearby adjectives.

Multi-headed attention



Feed-Forward Layer

- The feed-forward sublayer in the encoder and decoder is just a simple two-layer fully connected neural network;
- Instead of processing the whole sequence of embeddings as a single vector, it processes each embedding independently;
- The hidden size of the first layer needs to be four times the size of the embeddings, and a GeLU activation function is most commonly used.

Layer Normalization and Skip Connections

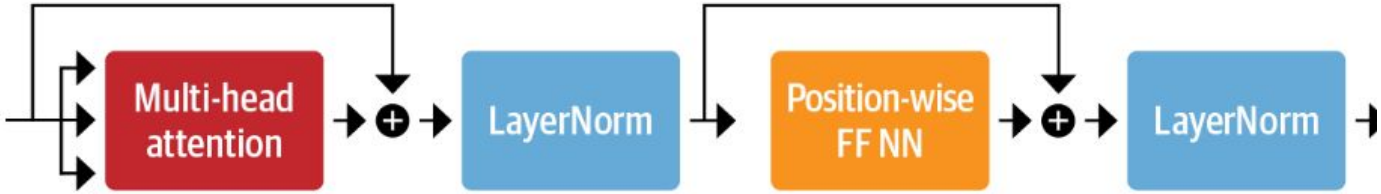
- As mentioned earlier, the Transformer architecture makes use of layer normalization and skip connections;
- The former normalizes each input in the batch to have zero mean and unity variance;
- Skip connections pass a tensor to the next layer of the model without processing and add it to the processed tensor.

Layer Normalization and Skip Connections

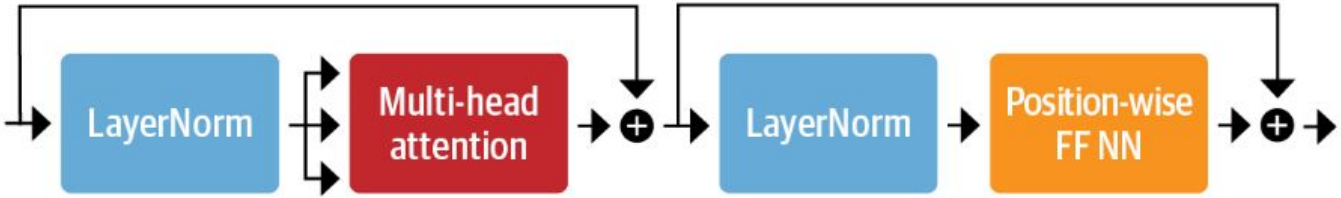
When it comes to placing the layer normalization in the encoder or decoder layers of a transformer, there are two main choices adopted in the literature:

- **Post layer normalization:** Based on the Transformer paper; it places layer normalization in between the skip connections. This arrangement is tricky to train from scratch as the gradients can diverge;
- **Pre layer normalization:** This is the most common arrangement found in the literature; it places layer normalization within the span of the skip connections. This tends to be much more stable during training.

Layer Normalization and Skip Connections



Post layer normalization



Pre layer normalization

Positional Embeddings

Positional embeddings are based on a simple, yet very effective idea: augment the token embeddings with a position-dependent pattern of values arranged in a vector.

Ways to do that:

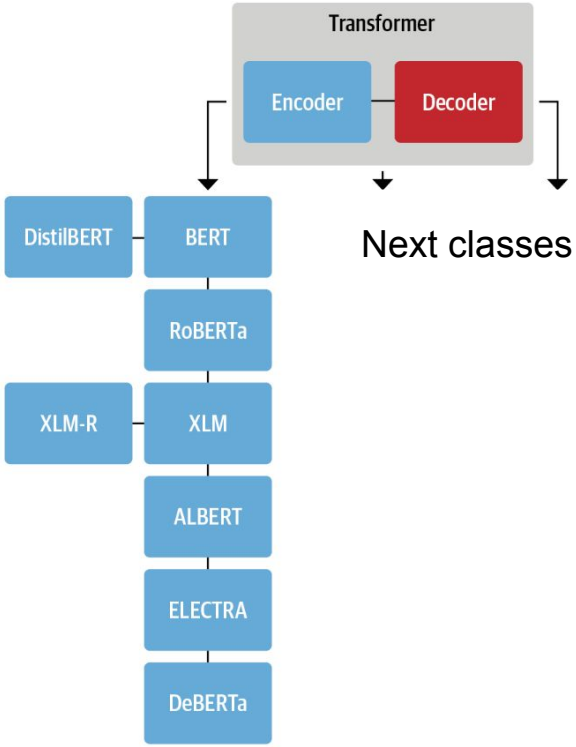
- **Learnable position embeddings:** works exactly the same way as the token embeddings, but using the position index instead of the token ID as input;
- **Absolute positional representation:** sine and cosine signals to encode the positions of the tokens;
- **Relative positional representations:** the attention mechanism itself is modified with additional terms that take the relative position between tokens into account.

Pre-training Preview

Before specialized tasks, the Encoder must undergo a massive learning phase called pre-training to understand the nuances of human language. This process is largely self-supervised, meaning the model learns from raw text without human labels.

- **The Core Concept:** The goal of pre-training is to create a "rich numerical representation" of any input sequence;
- **Masked Language Modeling (MLM):** A percentage of input tokens are hidden or replaced with a special [MASK] symbol;
- Once pre-trained, the model has a "General Understanding" of language.

The Encoder Tree of Life



Next classes

The Encoder Branch

The first encoder-only model based on the Transformer architecture was BERT. It outperformed all the state-of-the-art models on natural language understanding (NLU);

BERT model and its variants now a days:

- DistilBERT (knowledge distillation)
- RoBERTa (trained longer, larger batches and more data)
- XLM (ross-lingual Language Model)
- ALBERT (architecture and training more efficient)
- ModernBERT
- ModBERTBr
- BERTomelo
- …

Conclusion

- End-to-End Pipeline
  - Text → Tokenization → Embeddings → Attention → Output
  - Each step plays a critical role in how meaning is represented and processed.
- Key Components
  - Tokenization defines how text is split and understood.
  - Embeddings convert tokens into meaningful vector representations.
  - Self-attention enables contextual understanding across the sequence.

Conclusion

- Architectural Variants
  - Encoder-only → understanding tasks
  - Decoder-only → text generation
  - Encoder-decoder → sequence-to-sequence tasks
- Modern Extensions
  - Knowledge Distillation → smaller and faster models (e.g., DistilBERT)
  - Cross-lingual Models → multilingual understanding

Conclusion

Final Takeaway

- Transformers revolutionized NLP by enabling context-aware, scalable, and flexible models;
- The combination of tokenization, embeddings, and attention is the foundation of modern LLMs.

References

- TUNSTALL, Lewis; VON WERRA, Leandro; WOLF, Thomas. Natural language processing with transformers: building language applications with Hugging Face. O'Reilly Media, 2022.
- RASCHKA, Sebastian. Build a large language model (from scratch). Shelter Island: Manning Publications, 2024.
- VASWANI, Ashish; SHAZEER, Noam; PARMAR, Niki; USZKOREIT, Jakob; JONES, Llion; GOMEZ, Aidan N.; KAISER, Łukasz; POLOSUKHIN, Illia. Attention is all you need. 2017.
- DEVLIN, Jacob; CHANG, Ming-Wei; LEE, Kenton; TOUTANOVA, Kristina. BERT: pre-training of deep bidirectional transformers for language understanding. 2019.